

BOX : Homework 02

Question 1.

This algorithm is very slow : for 2 million bases and reads of length 100, we have to do in the order of 10^{16} string comparisons due to our $O(n^3)$ complexity. Because of this, the algorithm takes an unreasonable amount of time to run on more than about 1% of the given genome.

Question 2.

Let us consider the following reduction :

Let $S := \{s_1, \dots, s_n\}$ be an instance of the SCS problem. We define $G_S = (V, E, \rho)$ the directed weighted graph such that :

- $V = \{v_1, \dots, v_n, v_f\}$ where each vertex v_i represents the string s_i , and v_f is an extra vertex.
- $\forall i, j \in [1, n], (v_i, v_j) \in E$ and $\forall i \in [1, n], (v_i, v_f) \in E$: there is an edge between each pair of vertices representing a string, as well as an edge from every string vertex to v_f . Note that v_f is a well.
- $\forall i, j \in [1, n], \rho(v_i, v_j) = |s_i| - |\text{biggest_overlap}(s_i, s_j)|$: this way, $\rho(v_i, v_j)$ is the size of the smallest prefix we have to add to the word if we merge s_i with s_j .
- $\forall i \in [1, n], \rho(v_i, v_f) = |s_i|$

We then calculate a minimal weight Hamiltonian path. Since v_f is a well, the path ends at v_f . We can write the path as $H = v_{i_1} \dots v_{i_n} v_f$.

We can now reconstruct a smallest superstring by merging v_{i_1} with v_{i_2} , then successively merging every v_{i_k} in order.

Question 3.

Let S be an instance of SCS. Let us assume that $\forall u, v \in S, u \not\subseteq v$. We now show that the cost of a smallest Hamiltonian path in G_S is equal to the length of a shortest common superstring.

Lemma 1 : Merging is associative.

Let $s_{i_1}, s_{i_2}, s_{i_3}$ be words in S . Then $\text{merge}(\text{merge}(s_{i_1}, s_{i_2}), s_{i_3}) = \text{merge}(s_{i_1}, \text{merge}(s_{i_2}, s_{i_3}))$.

Proof : The only case in which this does not happen is if one word is a subword of another. Since we assume that this is not the case, merging is indeed associative. This can be extended to any number of words.

We now write $\text{merge}(s_{i_1}, \dots, s_{i_k})$ since we no longer have to specify the exact order in which we merge.

Lemma 2 : First, we show that $\forall k \in \mathbb{N}, \forall i_1, \dots, i_k \in [1, n]$, if $P = v_{i_1} \dots v_{i_k} v_f$ is a path of weight w , then the word $\text{merge}(s_{i_1}, \dots, s_{i_k})$ has length w .

Proof : We show this by induction on k .

Case $k = 1$:

The path is $P = v_i v_f$. The corresponding word is s_i . By definition, $\rho(P) = \rho(v_i, v_f) = |s_i|$.

Case $k > 1$:

Let us assume the property is true for all $k' < k$. Let us then show that it stays true for k .

Let $P = v_{i_1} \dots v_{i_k} v_f$. Then by our induction hypothesis, we know that $\rho(v_{i_2} \dots v_{i_k} v_f) = |\text{merge}(s_{i_2}, \dots, s_{i_k})|$.

Then, $\rho(P) = \rho(v_{i_1}, v_{i_2}) + \rho(v_{i_2} \dots v_{i_k} v_f) = |s_{i_1}| - |\text{biggest_overlap}(s_{i_1}, s_{i_2})| + |\text{merge}(s_{i_2}, \dots, s_{i_k})|$.

Additionally, $|\text{merge}(s_{i_1}, \dots, s_{i_k})| = |s_{i_1}| - \text{biggest_overlap}(s_{i_1}, s_{i_2}) + |\text{merge}(s_{i_2}, \dots, s_{i_k})|$.

Therefore, $\rho(P) = |\text{merge}(s_{i_1}, \dots, s_{i_k})|$.

So our Lemma holds.

Lemma 3 : There exists an order $i_1, \dots, i_n$ such that $\text{merge}(s_{i_1}, \dots, s_{i_n})$ is a smallest possible superstring.

Proof :

We know that a shortest superstring w must contain every substring at least once. Since no substring is a subword of another, w must be a succession of substrings, with possibly some overlap between substrings. The shortest way to do that is the result of a merge.

Theorem 4 : The cost of a smallest Hamiltonian path in G_S is equal to the length of a shortest common superstring in S .

Proof :

Since :

- A smallest possible superstring is the result of a merge of every substring s_i (Lemma 3),
- By definition, the result of every such merge is represented in G_S (Definitions, Lemma 2),
- Every Hamiltonian path in G_S represents the result of such a merge (Lemma 2),

We know that all smallest possible substrings are represented in G_S , and that the paths that represent them are the Hamiltonian paths of minimal weight.

So our reduction is correct.

Question 4.

N/A

Question 5.

Given that the TSP solver crashes my computer when used on instances made from more than a couple thousand bases, I can not provide an analysis of its performance. Nonetheless, that does make it seem significantly less efficient than the greedy algorithm.